ConvTranspose2d#

https://pytorch.org/docs/stable/generated/torch.nn.ConvTranspose2d.html

Description:

Applies a 2D transposed convolution operator over an input image composed of several input planes. This module can be seen as the gradient of Conv2d with respect to its input. It is also known as a fractionally-strided convolution or a deconvolution (although it is not an actual deconvolution operation as it does not compute a true inverse of convolution). For more information, see the visualizations here and the Deconvolutional Networks paper. This module supports TensorFloat32. On certain ROCm devices, when using float16 inputs this module will use different precision for backward. stride controls the stride for the cross-correlation. When stride > 1, ConvTranspose2d inserts zeros between input elements along the spatial dimensions before applying the convolution kernel. This zero-insertion operation is the standard behavior of transposed convolutions, which can increase the spatial resolution and is equivalent to a learnable upsampling operation.

Function Signature

```
class torch.nn.ConvTranspose2d(in_channels, out_channels,
kernel_size, stride=1, padding=0, output_padding=0,
groups=1, bias=True, dilation=1, padding_mode='zeros',
device=None, dtype=None)[source]#

Parameters

Shape:

Variables

forward(input, output_size=None)[source]#
```

Returns:

Tensor

Code Examples

```
>>> # With square kernels and equal stride
>>> m = nn.ConvTranspose2d(16, 33, 3, stride=2)
>>> # non-square kernels and unequal stride and with padding
>>> m = nn.ConvTranspose2d(16, 33, (3, 5), stride=(2, 1),
padding=(4, 2))
>>> input = torch.randn(20, 16, 50, 100)
>>> output = m(input)
>>> # exact output size can be also specified as an argument
>>> input = torch.randn(1, 16, 12, 12)
>>> downsample = nn.Conv2d(16, 16, 3, stride=2, padding=1)
>>> upsample = nn.ConvTranspose2d(16, 16, 3, stride=2,
padding=1)
>>> h = downsample(input)
>>> h.size()
torch.Size([1, 16, 6, 6])
>>> output = upsample(h, output_size=input.size())
>>> output.size()
torch.Size([1, 16, 12, 12])
```

Notes & Warnings

NOTE

Note The padding argument effectively adds dilation * (kernel_size - 1) - padding amount of zero padding to both sizes of the input. This is set so that when a Conv2d and a ConvTranspose2d are initialized with same parameters, they are inverses of each other in regard to the input and output shapes. However, when stride > 1, Conv2d maps multiple input shapes to the same output shape. output_padding is provided to resolve this ambiguity by effectively increasing the calculated output shape on one side. Note that output_padding is only used to find output shape, but does not actually add zero-padding to output.

NOTE

Note In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting torch.backends.cudnn.deterministic = True. See Reproducibility for more information.

Detailed Documentation

Documentation

Applies a 2D transposed convolution operator over an input image composed of several input planes.

This module can be seen as the gradient of Conv2d with respect to its input. It is also known as a fractionally-strided convolution or a deconvolution (although it is not an actual deconvolution operation as it does not compute a true inverse of convolution). For more information, see the visualizations here and the Deconvolutional Networks paper.

This module supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

stride controls the stride for the cross-correlation. When stride > 1, ConvTranspose2d inserts zeros between input elements along the spatial dimensions before applying the convolution kernel. This zero-insertion operation is the standard behavior of transposed convolutions, which can increase the spatial resolution and is equivalent to a learnable upsampling operation.

padding controls the amount of implicit zero padding on both sides for dilation * (kernel_size - 1) - padding number of points. See note below for details.

output_padding controls the additional size added to one side of the output shape. See note below for details.

dilation controls the spacing between the kernel points; also known as the à trous algorithm. It is harder to describe, but the link here has a nice visualization of what dilation does.

groups controls the connections between inputs and outputs. in_channels and out_channels must both be divisible by groups. For example,

At groups=1, all inputs are convolved to all outputs.

At groups=2, the operation becomes equivalent to having two conv layers side by side, each seeing half the input channels and producing half the output channels, and both subsequently concatenated.

At groups= in_channels, each input channel is convolved with its own set of filters (of size out_channelsin_channels\frac{\text{out_channels}}{\text{in_channels}}{\text{in_channels}}in_channelsout_channels).

The parameters kernel_size, stride, padding, output_padding can either be:

a single int – in which case the same value is used for the height and width dimensions

a tuple of two ints – in which case, the first int is used for the height dimension, and the second int for the width dimension

The padding argument effectively adds dilation * (kernel_size - 1) - padding amount of zero padding to both sizes of the input. This is set so that when a Conv2d and a ConvTranspose2d are initialized with same parameters, they are inverses of each other in regard to the input and output shapes. However, when stride > 1, Conv2d maps multiple input shapes to the same output shape. output_padding is provided to resolve this ambiguity by effectively increasing the calculated output shape on one side. Note that output_padding is only used to find output shape, but does not actually add zero-padding to output.

In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting torch.backends.cudnn.deterministic = True. See Reproducibility for more information.

in_channels (int) – Number of channels in the input image

out_channels (int) – Number of channels produced by the convolution

kernel_size (int or tuple) – Size of the convolving kernel

stride (int or tuple, optional) – Stride of the convolution. Default: 1

padding (int or tuple, optional) – dilation * (kernel_size - 1) - padding zero-padding will be added to both sides of each dimension in the input. Default: 0

output_padding (int or tuple, optional) – Additional size added to one side of each dimension in the output shape. Default: 0

groups (int, optional) – Number of blocked connections from input channels to output channels. Default: 1

bias (bool, optional) – If True, adds a learnable bias to the output. Default: True

dilation (int or tuple, optional) – Spacing between kernel elements. Default: 1

Input: (N,Cin,Hin,Win)(N, C_{in}, H_{in}, W_{in})(N,Cin,Hin,Win) or (Cin,Hin,Win)(C_{in}, H_{in}, W_{in})(Cin,Hin,Win)

Output: (N,Cout,Hout,Wout)(N, C_{out}, H_{out}, W_{out})(N,Cout,Hout,Wout) or (Cout,Hout,Wout)(C_{out}, H_{out}, W_{out})(Cout,Hout,Wout), where

weight (Tensor) – the learnable weights of the module of shape (in_channels,out_channelsgroups,(\text{in_channels}, \frac{\text{out_channels}}{\text{groups}},(in_channels,groupsout_channels, kernel_size[0],kernel_size[1])\text{kernel_size[0]}, \text{kernel_size[1]})kernel_size[0],kernel_size[1]). The values of these weights are sampled from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k) where k=groupsCout∗∏i=01kernel_size[i]k = \frac{groups}{C_\text{out} *

\prod_{i=0}^{1}\text{kernel_size}[i]}k=Cout∗∏i=01kernel_size[i]groups

bias (Tensor) – the learnable bias of the module of shape (out_channels) If bias is True, then the values of these weights are sampled from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k) where k=groupsCout∗∏i=01kernel_size[i]k = \frac{groups}{C_\text{out} * \prod_{i=0}^{1}\text{kernel_size}[i]}k=Cout∗∏i=01kernel_size[i]groups

Performs the forward pass.

input (Tensor) – The input tensor.

output_size (list[int], optional) – A list of integers representing the size of the output tensor. Default is None.
